Module 4

Graph Neural Networks

Module Overview

This module introduces **Graph Neural Networks (GNNs)**, the family of architectures designed to learn directly from graph-structured data. You will begin with the fundamentals of representing and reasoning about graphs, develop the general *message passing* framework underlying essentially all modern GNNs, and then study two of the most influential concrete architectures – the **Graph Convolutional Network (GCN)** and the **Graph Attention Network (GAT)** – in detail. By the end of this module, you should be able to implement the GCN and GAT propagation rules from first principles, explain why permutation invariance is central to GNN design, and describe how sheaf-theoretic ideas generalize the graph Laplacian.

Learning Outcomes

By the end of this module, students should be able to:

1. **Explain** the fundamental concepts of graph-structured data.
2. **Implement** Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs).
3. **Analyze** the strengths and limitations of Graph Convolutional Networks and Graph Attention Networks.
4. **Evaluate** the principles of Sheaf Neural Networks.

1 Introduction to Graph-Structured Data

1.1 What Is a Graph?

Definition

A **graph** is a pair $G = (V, E)$, where $V = \{v_1, \ldots, v_n\}$ is a set of n **nodes** (or **vertices**), and $E \subseteq V \times V$ is a set of **edges** connecting pairs of nodes. A graph may additionally carry:

- **Node features:** a feature vector $x_v \in \mathbb{R}^d$ for every node $v \in V$, collected into a matrix $X \in \mathbb{R}^{n \times d}$.
- **Edge features:** a feature vector for every edge (e.g., edge type, weight, distance).
- **Graph-level features:** a global feature vector describing the graph as a whole.

Graphs are described as **directed** if edges (u, v) have a direction (distinct from (v, u)), or **undirected** otherwise; as **weighted** if each edge carries a real-valued weight, or **unweighted** otherwise; and as **homogeneous** if all nodes/edges are of a single type, or **heterogeneous** if multiple node/edge types coexist (e.g., a knowledge graph with "person," "movie," and "genre" node types connected by "acted in" and "belongs to" edge types).

Definition

The **adjacency matrix** $A \in \{0, 1\}^{n \times n}$ (or $\mathbb{R}^{n \times n}$ if weighted) of a graph encodes its connectivity: $A_{ij} = 1$ if there is an edge from node i to node j (and A_{ij} equals the edge weight for weighted graphs), and $A_{ij} = 0$ otherwise. For an undirected graph, A is symmetric.
The **degree matrix** $D \in \mathbb{R}^{n \times n}$ is a diagonal matrix with $D_{ii} = \sum_j A_{ij}$, the degree (number of neighbors, or sum of incident edge weights) of node i.
The (combinatorial) **graph Laplacian** is defined as:

$$L = D - A$$

Key Idea

The graph Laplacian L is the discrete analogue of the continuous Laplacian (second-derivative / divergence-of-gradient) operator from calcu-

lus. Just as the continuous Laplacian measures how much a function at a point differs from its local average, $(Lx)_i = \sum_j A_{ij}(x_i - x_j)$ measures how much node i's value differs from a (degree-weighted) average of its neighbors' values. This makes L the natural operator for defining notions of *smoothness* over a graph, and it underlies both the spectral theory in Section 3 and the sheaf-theoretic generalization in Section 5.

1.2 Why Graphs Require New Architectures

Standard deep learning architectures rely on strong assumptions about input structure that graphs violate:

- **CNNs** assume inputs live on a regular grid, where every "pixel" has a fixed, ordered set of neighbors (e.g., the 8 surrounding pixels), allowing a single shared convolutional kernel to be applied identically everywhere.
- **RNNs/Transformers** assume inputs form a sequence (or, for Transformers, at least a fixed, orderable set of tokens).
- **Graphs** have *no fixed ordering of nodes*, and different nodes can have wildly different, variable numbers of neighbors. A node's neighbors have no inherent order either (unlike, say, "the pixel to the left").

Key Idea

Any architecture operating on graphs must be **permutation invariant** (for graph-level outputs) or **permutation equivariant** (for node-level outputs): if we relabel/reorder the nodes of a graph (and correspondingly permute the rows/columns of A and rows of X), the graph itself has not changed, so a graph-level prediction must be identical, and node-level predictions must simply be permuted in the same way. This single requirement – rather than any specific architecture – is the central design constraint that defines the entire field of GNNs, and it rules out simply "flattening" a graph's adjacency matrix and feeding it to a standard MLP (which would treat different node orderings as different inputs).

Definition

Formally, a function f operating on a graph is **permutation equiv-**

ariant if, for every permutation matrix P:

$$f(PX, PAP^\top) = Pf(X, A)$$

and **permutation invariant** if:

$$f(PX, PAP^\top) = f(X, A)$$

Node-level tasks (e.g., node classification) require equivariance; graph-level tasks (e.g., graph classification) require invariance, typically achieved by applying an equivariant function followed by a permutation-invariant **readout** (e.g., sum, mean, or max over all nodes).

1.3 Common Graph Learning Tasks

- **Node classification:** Predict a label for each node (e.g., classifying papers into research topics in a citation network), typically in a *transductive* setting where the whole graph, including test nodes' features and connectivity (but not their labels), is visible during training.
- **Link prediction:** Predict whether an edge should exist between two nodes (e.g., recommending new friend connections, or predicting missing protein-protein interactions).
- **Graph classification / regression:** Predict a single label or value for an entire graph (e.g., predicting a molecule's toxicity or solubility from its atomic structure).
- **Graph generation:** Generate new graphs with desired properties (e.g., generating novel candidate drug molecules).

1.4 Real-World Examples

Domain	**Nodes**	**Edges**	**Typical task**
Social networks	Users	Friendships/follows	Node classification, link prediction
Molecules	Atoms	Chemical bonds	Graph classification (property prediction)
Citation networks	Papers	Citations	Node classification
Knowledge graphs	Entities	Typed relations	Link prediction
Road networks	Intersections	Road segments	ETA prediction, routing

2 Graph Neural Networks: The Message Passing Framework

Almost every modern GNN architecture – including GCN and GAT, studied in detail below – is an instance of a single general computational pattern known as **Message Passing** (or, equivalently, **neighborhood aggregation**).

2.1 The Message Passing Recipe

Definition

A **Message Passing Neural Network (MPNN)** layer updates every node's representation by (1) computing a **message** from each of its neighbors, (2) **aggregating** these messages with a permutation-invariant function, and (3) **updating** the node's own representation using the aggregated message. Formally, for layer k:

$$m_v^{(k)} = \text{AGGREGATE}^{(k)}\Big(\big\{\, \text{MESSAGE}^{(k)}(h_v^{(k-1)}, h_u^{(k-1)}, e_{uv}) \;:\; u \in \mathcal{N}(v) \,\big\}\Big)$$

$$h_v^{(k)} = \text{UPDATE}^{(k)}\big(h_v^{(k-1)}, m_v^{(k)}\big)$$

where $\mathcal{N}(v)$ is the set of neighbors of node v, $h_v^{(k)}$ is node v's representation (*embedding*) after k layers, $h_v^{(0)} = x_v$ (the input node features), and e_{uv} is the (optional) feature of the edge between u and v.

Key Idea

Because AGGREGATE is required to be a **permutation-invariant** function of the (unordered) set of incoming messages – typically sum, mean, or max – the entire MPNN layer is automatically permutation equivariant: relabeling the nodes simply relabels which message goes where, without changing the computation itself. This is precisely how GNNs satisfy the structural requirement identified in Section 1.2, and it is the single unifying idea beneath GCN, GAT, GraphSAGE, and most other GNN variants – they differ from one another primarily in their specific choice of MESSAGE, AGGREGATE, and UPDATE functions.

2.2 Stacking Layers and the Receptive Field

After k layers of message passing, node v's representation $h_v^{(k)}$ has aggregated information from every node within k hops in the graph – its k-hop neighborhood. This is directly analogous to how stacking convolutional layers in a CNN enlarges the receptive field of each unit.

Example 1. *Consider a simple chain graph $A - B - C - D - E$. After one message-passing layer, node C's representation incorporates information from B and D (its direct neighbors). After two layers, C's representation has indirectly incorporated information from A and E as well (since B's layer-1 representation already included A, and D's included E). In general, k layers are required for information to travel a graph distance of k.*

2.3 Readout for Graph-Level Tasks

For graph classification/regression, after K layers of message passing produce final node embeddings $h_v^{(K)}$ for every node, a **readout** (or *pooling*) function combines all node embeddings into a single graph-level embedding:

$$h_G = \text{READOUT}\big(\{h_v^{(K)} : v \in V\}\big)$$

Common choices include sum, mean, or max pooling over nodes, or more sophisticated learned pooling mechanisms (e.g., attention-based pooling, or hierarchical graph pooling that progressively coarsens the graph).

Key Idea

Just as AGGREGATE must be permutation-invariant within a node's neighborhood, READOUT must be permutation-invariant over the *whole graph's* nodes, for exactly the same reason: the graph-level output must not depend on how the nodes happen to be numbered.

2.4 Fundamental Limitations: Over-smoothing and Over-squashing

- **Over-smoothing:** As more message-passing layers are stacked, node representations tend to converge toward similar (indistinguishable) values, since repeatedly averaging with neighbors acts like repeated low-pass filtering. This is analogous to what happens if you were to apply a heat-diffusion / blurring operation to an image many times – eventually every pixel converges to the same value. In practice, this limits most standard

GNNs to a small number of layers (often just 2–4), which in turn limits their effective receptive field.

- **Over-squashing:** Information from an exponentially growing number of distant nodes must be compressed into a single fixed-size vector as it propagates toward a target node, creating an information bottleneck that makes it difficult for long-range dependencies to be preserved, particularly across graph regions connected by a small number of edges (bottlenecks in the graph's topology).

Key Idea

Over-smoothing and over-squashing are two sides of the same underlying tension: a GNN needs enough layers to reach distant, potentially relevant nodes, but adding layers degrades the distinctiveness of node representations and compresses ever more information through narrow graph bottlenecks. Much of the ongoing research in GNN architecture design (including the Sheaf Neural Networks discussed in Section 5) is aimed directly at mitigating one or both of these problems.

3 Graph Convolutional Networks (GCN)

The **Graph Convolutional Network** (Kipf & Welling, 2017) is one of the most influential and widely used GNN architectures, arising as a first-order approximation of earlier spectral graph convolution methods.

3.1 Spectral Motivation

Key Idea

The name "graph convolution" originates from an analogy with the Convolution Theorem in signal processing: convolution in the spatial domain corresponds to multiplication in the frequency domain. For graphs, the appropriate notion of "frequency" comes from the eigenvectors of the graph Laplacian $L = D - A$ (the **graph Fourier basis**), and a **spectral graph convolution** is defined as an operation that multiplies a signal's graph-Fourier-transformed representation by some filter, then transforms back. Early spectral GNNs (e.g., ChebNet) approximated arbitrary spectral filters using Chebyshev polynomials of L, which are K-localized (a degree-K polynomial filter only involves information from within K hops) but still require an eigendecomposition-free but computationally involved recursive formulation.

GCN simplifies this framework substantially by restricting the Chebyshev polynomial filter to a first-order ($K = 1$) approximation and tying/simplifying its parameters, yielding an extremely simple and efficient propagation rule that in practice performs very well.

3.2 The GCN Propagation Rule

Definition

Given node feature matrix $H^{(l)} \in \mathbb{R}^{n \times d_l}$ at layer l (with $H^{(0)} = X$), the **GCN layer** computes:

$$H^{(l+1)} = \sigma\left(\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

where:

- $\hat{A} = A + I$ is the adjacency matrix with added **self-loops** (so that a node's own previous-layer representation is included when aggregating).

- $\hat{D}$ is the degree matrix of $\hat{A}$ (i.e., $\hat{D}_{ii} = \sum_j \hat{A}_{ij}$).
- $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ is a learned weight matrix, shared across all nodes.
- σ is a nonlinearity (typically ReLU).

Let us unpack why each piece is needed:

1. **Self-loops ($\hat{A} = A + I$):** Without self-loops, a node's aggregation would only involve its neighbors, entirely discarding the node's own previous representation. Adding self-loops ensures each node's update includes (an appropriately weighted share of) its own features – this is sometimes called the **renormalization trick**.

2. **Symmetric normalization ($\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}$):** Using the raw adjacency matrix $\hat{A}$ alone would cause high-degree nodes' representations to have much larger magnitude after aggregation (since they sum over more neighbors) than low-degree nodes, causing instability during training. Symmetric normalization rescales each entry $\hat{A}_{ij}$ by $\frac{1}{\sqrt{\hat{D}_{ii}}\sqrt{\hat{D}_{jj}}}$, so that both the source and target node's degree are accounted for – keeping the propagated signal at a numerically stable, comparable scale regardless of degree, and preventing the operator's largest eigenvalues from exploding.

3. **Shared weight matrix $W^{(l)}$:** As in a CNN, the same weight matrix is applied identically to every node, keeping the number of parameters independent of graph size and providing the same permutation-equivariance guarantee established in Section 2.

Key Idea

A single GCN layer can be understood, at the level of an individual node v, as: *take a (degree-normalized) average of your own and your neighbors' previous-layer representations, linearly transform the result with a shared weight matrix, then apply a nonlinearity.* This is a highly efficient, simple instantiation of the general message-passing recipe from Section 2, where MESSAGE is a normalized copy of the neighbor's features, AGGREGATE is a (normalized) sum, and UPDATE applies $W^{(l)}$ and σ.

3.3 Node-Level View of the Propagation Rule

Writing the matrix equation out per node makes the neighborhood-averaging interpretation explicit:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\hat{d}_v \hat{d}_u}} W^{(l)} h_u^{(l)}\right)$$

where $\hat{d}_v = \hat{D}_{vv}$ is node v's degree after adding the self-loop.

Example 2. *Consider a small graph with 3 nodes: A connected to B, and B connected to C (a chain, no edge between A and C). With self-loops added, node B has degree $\hat{d}_B = 3$ (itself, A, C), while A and C each have degree $\hat{d}_A = \hat{d}_C = 2$ (themselves and B). For node A's update, the normalization coefficient for its message from itself is $\frac{1}{\sqrt{2\times 2}} = 0.5$, and from B is $\frac{1}{\sqrt{2\times 3}} \approx 0.408$. Node B, having a higher degree, contributes a slightly* smaller-*magnitude normalized message to its neighbors than a low-degree node would, illustrating how symmetric normalization down-weights the influence of high-degree "hub" nodes relative to what an unnormalized sum would give.*

3.4 Practical Considerations and Limitations

- **Fixed, uniform neighbor weighting:** Every neighbor's contribution is weighted purely by the (fixed, data-independent) degree-based normalization – the model cannot learn to pay more attention to some neighbors than others based on their content. This is the primary limitation that Graph Attention Networks (Section 4) address.

- **Requires the full graph structure:** The original GCN formulation processes the entire graph's normalized adjacency matrix at once, making it naturally suited to the transductive node classification setting, though sampling-based extensions (e.g., GraphSAGE) adapt the same underlying idea to large-scale or inductive settings.

- **Susceptible to over-smoothing:** As discussed in Section 2.4, stacking many GCN layers causes node representations to converge, so GCNs are typically shallow (2–3 layers) in practice.

4 Graph Attention Networks (GAT)

The **Graph Attention Network** (Veličković et al., 2018) replaces GCN's fixed, degree-based neighbor weighting with **learned, content-dependent attention weights**, directly borrowing the attention mechanism studied in Module 3 and adapting it to the graph setting.

4.1 Attention Coefficients

Definition

For each edge (u, v) (with $u \in \mathcal{N}(v)$), GAT computes an unnormalized **attention coefficient**:

$$e_{vu} = \text{LeakyReLU}\Big(a^\top \big[Wh_v \,\|\, Wh_u\big]\Big)$$

where $W \in \mathbb{R}^{d' \times d}$ is a shared linear transformation applied to every node's features, $\|$ denotes vector concatenation, and $a \in \mathbb{R}^{2d'}$ is a learned attention vector (equivalently, a single-layer feed-forward network mapping the concatenated pair of transformed features to a scalar score).

These coefficients are then normalized via softmax over each node's neighborhood:

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{u' \in \mathcal{N}(v)} \exp(e_{vu'})}$$

and used to compute node v's updated representation as an attention-weighted combination of its (transformed) neighbors:

$$h'_v = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu} \, W h_u\right)$$

Key Idea

Compare this directly to GCN's propagation rule: GCN uses the *fixed* coefficient $\frac{1}{\sqrt{\hat{d}_v \hat{d}_u}}$, determined purely by graph structure (node degrees) and identical for every input; GAT instead learns α_{vu} as a function of the actual node *features* h_v, h_u, allowing the model to assign different importance to different neighbors depending on their content, and allowing this importance to vary across different graphs or differ-

ent feature values on the *same* graph structure. This closely mirrors the distinction between self-attention (content-based, learned weighting) and a fixed uniform or positionally-determined weighting scheme in sequence models.

4.2 Multi-Head Attention in GAT

Exactly as in the Transformer (Module 3), GAT typically uses **multiple attention heads** to stabilize learning and allow the model to attend to different aspects of the neighborhood simultaneously:

$$h'_v = \Big\|_{k=1}^{K} \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(k)} W^{(k)} h_u\right)$$

where K heads' outputs are **concatenated** (denoted $\|$) at intermediate layers. At the *final* layer, outputs are typically **averaged** rather than concatenated (to keep the final output dimension fixed and interpretable, e.g., matching the number of classes):

$$h'_v = \sigma\left(\frac{1}{K}\sum_{k=1}^{K}\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(k)} W^{(k)} h_u\right)$$

4.3 Worked Example: Computing Attention Coefficients

Example 3. *Suppose node v has two neighbors, u_1 and u_2, and after applying W, the raw scores (before softmax) are $e_{vu_1} = 2.0$ and $e_{vu_2} = 0.5$. The normalized attention weights are:*

$$\alpha_{vu_1} = \frac{e^{2.0}}{e^{2.0} + e^{0.5}} = \frac{7.389}{7.389 + 1.649} \approx 0.818, \qquad \alpha_{vu_2} \approx 0.182$$

Node v's update is therefore dominated by u_1's (transformed) features, with a smaller contribution from u_2 – a data-dependent weighting that GCN's fixed degree-based normalization could not express (GCN would weight both neighbors purely by $1/\sqrt{\hat{d}_v \hat{d}_{u_i}}$, regardless of the actual content of h_{u_1} or h_{u_2}).

4.4 GATv2: Fixing Static Attention

A subtle but important limitation of the original GAT was identified by Brody et al. (2021): because the LeakyReLU nonlinearity is applied *after* the linear combination $a^\top[Wh_v \| Wh_u]$, and a is a single shared vector, the *ranking* of attention scores across different neighbors u of a fixed query node v turns out to be independent of v's own features in certain settings – a phenomenon termed **static attention**, which limits the expressivity of the attention mechanism. **GATv2** fixes this by reordering the operations, applying the learned vector a *after* the nonlinearity:

$$e_{vu} = a^\top \mathrm{LeakyReLU}\Big(W\big[h_v \,\|\, h_u\big]\Big)$$

This seemingly small reordering restores full (*dynamic*) attention expressivity, where the relative ranking of a node's neighbors can genuinely depend on the query node's own features, and GATv2 has since become a common drop-in replacement for the original GAT layer.

4.5 GCN vs. GAT: Summary Comparison

	GCN	**GAT**
Neighbor weighting	Fixed, determined by graph degree ($\hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}$)	Learned, determined by node content (softmax attention)
Adapts to features?	No – same weighting for any feature values on a fixed graph	Yes – weighting changes with node features
Parameter count	Lower (one weight matrix per layer)	Higher (weight matrix + attention vector per head, multiple heads)
Interpretability	Weighting is transparent but content-agnostic	Attention weights offer a direct view into neighbor importance

5 Sheaf Neural Networks

Standard GNNs – including GCN and GAT – share an implicit assumption that turns out to be a significant limitation: they assume connected nodes should end up with *similar* representations. **Sheaf Neural Networks** (Bodnar et al., 2022) generalize the mathematical foundation of GNNs using tools from **cellular sheaf theory**, directly addressing this limitation.

5.1 Motivation: The Homophily Assumption

Key Idea

Message passing, as defined in Section 2, and GCN's averaging in particular, implicitly assumes **homophily**: the idea that connected nodes tend to be similar, and that averaging a node's representation with its neighbors' should therefore produce a more useful representation. Many important real-world graphs are, however, **heterophilic**: connected nodes are often *dissimilar* (e.g., in a predator-prey food web, an edge connects two *different* kinds of organisms; in some fraud-detection networks, fraudulent nodes deliberately connect to legitimate-looking ones). On heterophilic graphs, standard GNNs' tendency to smooth node representations toward their neighbors' values actively *destroys* useful signal rather than enhancing it, and this same tendency is closely related to the over-smoothing problem from Section 2.4.

5.2 From Graphs to Cellular Sheaves

The key idea of a **cellular sheaf** on a graph is to move beyond a single scalar or vector value per node, and instead track how each node's information should be *translated* before being compared to each of its neighbors – rather than assuming all nodes share one common, global coordinate system (frame of reference).

Definition

A **cellular sheaf** $\mathcal{F}$ on a graph $G = (V, E)$ consists of:

- A vector space $\mathcal{F}(v)$, called the **stalk** at node v, for every $v \in V$ (and similarly a stalk $\mathcal{F}(e)$ for every edge e).
- For every incident (node, edge) pair $v \triangleleft e$, a linear **restriction map** $\mathcal{F}_{v \triangleleft e} : \mathcal{F}(v) \to \mathcal{F}(e)$.

A choice of vector $x_v \in \mathcal{F}(v)$ for every node is called a 0-**cochain**; the sheaf is **consistent** (the cochain is a **global section**) at edge $e = (v, u)$ if $\mathcal{F}_{v \trianglelefteq e}(x_v) = \mathcal{F}_{u \trianglelefteq e}(x_u)$ – i.e., both endpoints agree once translated into the edge's shared stalk.

Key Idea

The restriction maps play the role of "translation dictionaries" between each node's local frame of reference and the shared meaning at an edge. If every stalk is $\mathbb{R}$ (a single scalar) and every restriction map is the identity, disagreement across an edge reduces exactly to $x_v - x_u$, and the resulting sheaf Laplacian (defined next) reduces exactly to the standard graph Laplacian L from Section 1.1 – so an ordinary graph is precisely the special case of a cellular sheaf with trivial (identity) restriction maps. Allowing *non-trivial*, and crucially *learned*, restriction maps is what gives Sheaf Neural Networks their extra expressive power.

5.3 The Sheaf Laplacian

Definition

Collecting all restriction maps into a block **coboundary matrix** B (analogous to the incidence matrix of a graph, but built from the linear restriction maps rather than simple ± 1 entries), the **sheaf Laplacian** is defined as:

$$L_{\mathcal{F}} = B^\top B$$

For a 0-cochain $x = (x_v)_{v \in V}$, the quadratic form:

$$x^\top L_{\mathcal{F}} x = \sum_{e=(v,u) \in E} \left\| \mathcal{F}_{v \trianglelefteq e}(x_v) - \mathcal{F}_{u \trianglelefteq e}(x_u) \right\|^2$$

measures the total disagreement across all edges, generalizing the standard graph Laplacian's quadratic form $x^\top L x = \sum_{(v,u) \in E} (x_v - x_u)^2$.

Key Idea

This generalization is precisely what allows Sheaf Neural Networks to represent heterophily naturally: with a well-chosen (typically *learned*) restriction map, two connected nodes can hold genuinely different raw feature vectors $x_v \neq x_u$ while still being perfectly "consistent" ac-

cording to the sheaf (i.e., contributing zero to the disagreement term), *provided* the learned restriction maps correctly capture the systematic relationship/transformation between them. A standard GNN, using the trivial-sheaf assumption, has no such flexibility: it can only reduce disagreement by making x_v and x_u literally similar, which is exactly the mechanism that fails on heterophilic graphs.

5.4 Sheaf Diffusion and Sheaf Neural Network Layers

Just as GCN can be derived from (an approximation to) the diffusion equation $\frac{dX}{dt} = -LX$ using the standard graph Laplacian, a **Sheaf Neural Network** layer is derived from the analogous **sheaf diffusion** equation using the sheaf Laplacian $L_{\mathcal{F}}$:

$$\frac{dX}{dt} = -L_{\mathcal{F}}X$$

Discretizing this with a single (Euler) step, and adding a learned per-node linear transformation W and nonlinearity (exactly mirroring how GCN was derived), yields a propagation rule of the same general shape as GCN:

$$X^{(l+1)} = \sigma\Big(X^{(l)} - \Delta t\, L_{\mathcal{F}}^{(l)} X^{(l)} W^{(l)}\Big)$$

where, crucially, the restriction maps that determine $L_{\mathcal{F}}^{(l)}$ are themselves typically **learned** (often as a function of the two endpoint nodes' features), allowing the model to discover, per edge, what kind of "translation" should relate its two endpoints – rather than assuming, as GCN and GAT implicitly do, that the appropriate relationship is always "become more similar."

Key Idea

Learned, non-trivial restriction maps also directly mitigate over-smoothing: whereas repeated application of the standard graph Laplacian's diffusion process drives all connected node values toward a single shared constant (the well-known over-smoothing failure mode), sheaf diffusion converges instead toward the space of *global sections* (cochains consistent with the learned restriction maps) – a potentially much richer space than "all nodes equal," since a global section can hold genuinely different (but consistently related, via the restriction maps) values at different nodes.

5.5 Summary: How Sheaf Neural Networks Generalize GCN and GAT

	Standard GNN (GCN/GAT)	**Sheaf Neural Network**
Underlying operator	Graph Laplacian $L = D - A$	Sheaf Laplacian $L_{\mathcal{F}} = B^\top B$
Implicit assumption	Connected nodes should become similar (homophily)	Connected nodes should be *consistent under learned translation*
Handles heterophily?	Poorly – averaging degrades signal when neighbors are dissimilar	Well – learned restriction maps encode non-identity neighbor relationships
Special case relation	–	Reduces to standard GNN when restriction maps are identity

REFERENCES

- Prince, Simon JD. Understanding deep learning. MIT press, 2023.
- Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning (Vol. 1, No. 2, pp. 1-800). Cambridge: MIT press. https://www.deeplearningbook.org/
- Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Vol. 1. Cham, Switzerland: Springer, 2024.
- Torralba, A., Isola, P., & Freeman, W. T. (2024). Foundations of computer vision. MIT Press.